Tutorial 3: Use Case Modelling

Today we will learn how to show software requirements using a simple diagram.

Main example: Library Management System.

What are we doing today?

In Lab 1, we wrote requirements in an SRS.

Now we will convert some of those requirements into a picture.

This picture is called a Use Case Diagram.

It shows WHO uses the system and WHAT they want to do.

Why do we need a Use Case Diagram?

A long SRS can be difficult to understand quickly.

A diagram gives a simple visual view of the system.

It helps students, developers and customers discuss the same requirements.

It can help find missing or unclear requirements before coding starts.

Simple real-life example

Who uses an ATM? -> Customer.

What does the customer do? -> Withdraw Cash, Check Balance, Deposit Money.

Customer is an ACTOR.

Withdraw Cash is a USE CASE.

This same idea is used for the Library Management System.

What is an Actor?

An actor is a person or another external system that interacts with our software.

Actors are shown outside the system box.

Library examples: Member and Librarian.

Important: an actor is a role, not every noun in the problem.

What is a Use Case?

A use case is a service or task provided by the software.

Use cases are normally written as action names.

Examples: Search Book, Reserve Book, Issue Book, Return Book.

Use cases are drawn inside the system boundary.

What is the System Boundary?

Draw one rectangle and write 'Library Management System' on it.

Actors stay OUTSIDE the rectangle.

Use cases stay INSIDE the rectangle.

The boundary helps us separate the software from its users/environment.

What is Association?

A simple line between an actor and a use case is called an association.

Example: Member ----- Search Book.

It means the Member interacts with the Search Book function.

Do not connect an actor to a function that the actor does not use.

Understanding <<include>>

Use <<include>> when one use case always needs another use case.

Simple example: Issue Book may include Check Member Details.

The included task is a required part of the main task.

Do not use <<include>> everywhere. Use it only when it is really needed.

Understanding <<extend>>

An optional or conditional action

Use <<extend>> when extra behaviour happens only in some situations.

Example: Pay Fine may extend Return Book when a fine is due.

If there is no fine, Pay Fine is not performed.

Remember: include = required; extend = optional/conditional.

Library Management System example

Actor 1: Member -> Search Book, View Availability, Reserve Book.

Actor 2: Librarian -> Issue Book, Return Book, Maintain Book Records.

Draw the Library Management System rectangle.

Put the use cases inside and actors outside.

Connect each actor only to the use cases they use.

How to identify actors and use cases

Use your SRS instead of guessing

Step 1: Read one requirement.

Step 2: Ask 'Who is using the system?' -> actor.

Step 3: Ask 'What does this person want to do?' -> use case.

Step 4: Repeat for the other requirements.

Step 5: Remove duplicates and use clear action names.

Common mistakes

Do not put Member or Librarian inside the system boundary.

Do not put use cases outside the boundary.

Do not treat every noun as an actor.

Do not use <<include>> and <<extend>> just to make the diagram look complex.

The diagram must match the SRS requirements.

Quick classroom check

Is 'Member' an actor or a use case? -> Actor.

Is 'Search Book' an actor or a use case? -> Use case.

Where does an actor stay? -> Outside the system boundary.

Where does a use case stay? -> Inside the system boundary.

Which is normally conditional: include or extend? -> Extend.

Connection to today's Lab

You will use the Library Management System SRS from Lab 1.

You will identify actors and use cases.

You will draw one complete Use Case Diagram.